CIS 3300 · GENERICS AND COLLECTIONS, PART 2 OF 2

Collections

Lists, sets, maps, and queues, and how to choose among them

Since the first weeks of the course you have used `ArrayList`. Java has a whole family of collections, each built for a different job, and choosing the right one makes a program shorter, faster, and clearer. This part covers the main interfaces and the classes that implement them, the rules that hash and tree collections depend on, and the mistakes that cause the most common collection exceptions. For reading, see Core Java, Volume I, Chapter 9. In class, the Speedrun Leaderboard example chooses its collections in Checkpoint 4.

A note about these notes. I wrote these notes with help from AI, from the examples and explanations I use in class. The AI drafted much of the text and code, and I review and revise them. Every example was compiled and run, and every output and error message shown is real; long stack traces are shortened. If something looks wrong, does not match what you see in Eclipse, or contradicts the book, tell me. The textbook is the tiebreaker, and the compiler beats us both. These notes do not replace the assigned chapters.

1. The Examples

The examples use one small record and a playlist of seven songs. The songs and artists are invented.

```
public record Song(String title, String artist, int seconds) { }
```

Table 1: The playlist used in the examples, in its original order.

Title	Artist	Seconds
Midnight Drive	Neon Harbor	254
Paper Moons	The Lanterns	198
Slow Burn	Neon Harbor	312
Northern Lights	Ada Vale	287
Echo Park	The Lanterns	221
Glass Garden	Ada Vale	345
Static	Neon Harbor	176

`Playlist.songs()` returns these songs as a `List<Song>` that cannot be changed.

2. The Collection Interfaces

Java's collections are organized as interfaces, each with several classes that implement it. Table 2 lists the ones used most.

Table 2: The main collection interfaces.

Interface	What it holds	Common classes
`List`	Elements in order, by position, duplicates allowed	`ArrayList`
`Set`	Elements with no duplicates	`HashSet`, `LinkedHashSet`, `TreeSet`
`Map`	Keys, each with one value	`HashMap`, `LinkedHashMap`, `TreeMap`
`Queue`, `Deque`	Elements waiting their turn, taken from one end or both	`ArrayDeque`

`List`, `Set`, and `Queue` all extend the `Collection` interface, so they share methods such as `add`, `remove`, `contains`, `size`, and `isEmpty`. A `Map` is not a `Collection`, because it holds pairs rather than single elements.

Declare a variable with the interface type, and create it with the class:

```
List<Song> queue = new ArrayList<>();
Map<String, Integer> plays = new TreeMap<>();
```

The rest of the program then depends only on what a `List` or a `Map` can do, and switching to a different class means changing one word.

3. Lists

A list keeps its elements in the order you put them, finds them by position, and allows duplicates.

```
List<String> queue = new ArrayList<>();
queue.add("Static");
queue.add("Echo Park");
queue.add(0, "Slow Burn");          // insert at the front
queue.add("Static");                // duplicates are allowed
System.out.println(queue + ", size " + queue.size());
System.out.println(queue.get(1) + ", index of Static: " + queue.indexOf("Static"));
queue.set(1, "Paper Moons");        // replace the element at index 1
System.out.println(queue);
```

```
[Slow Burn, Static, Echo Park, Static], size 4
Static, index of Static: 1
[Slow Burn, Paper Moons, Echo Park, Static]
```

`ArrayList` is the right list almost every time. Java also has `LinkedList`, which is faster only for adding and removing at the ends of very long lists; use an `ArrayDeque` for that instead (Section 10).

A trap: `remove` with a `List<Integer>`

`List` has two `remove` methods: `remove(int index)` and `remove(Object value)`. With a list of `Integer`s, an `int` argument picks the first one:

```
List<Integer> lengths = new ArrayList<>(List.of(254, 198, 312));
lengths.remove(1);                          // removes index 1
System.out.println(lengths);
lengths.remove(Integer.valueOf(254));       // removes the value 254
System.out.println(lengths);
```

```
[254, 312]
[312]
```

4. Sets

A set holds each element once. The three common set classes differ only in the order in which they give their elements back:

```
List<String> titles = List.of("Static", "Echo Park", "Slow Burn", "Static",
        "Glass Garden", "Paper Moons", "Echo Park", "Northern Lights");
Set<String> hash = new HashSet<>(titles);
Set<String> linked = new LinkedHashSet<>(titles);
Set<String> tree = new TreeSet<>(titles);
System.out.println("HashSet:       " + hash);
System.out.println("LinkedHashSet: " + linked);
System.out.println("TreeSet:       " + tree);
System.out.println("add again: " + tree.add("Static"));
```

```
HashSet:       [Echo Park, Slow Burn, Northern Lights, Paper Moons, Static, Glass Garden]
LinkedHashSet: [Static, Echo Park, Slow Burn, Glass Garden, Paper Moons, Northern Lights]
TreeSet:       [Echo Park, Glass Garden, Northern Lights, Paper Moons, Slow Burn, Static]
add again: false
```

The eight titles became six in every set. A `HashSet` gives them back in no useful order, and Java does not promise even that order will stay the same. A `LinkedHashSet` keeps the order in which each was first added. A `TreeSet` keeps them sorted. Adding an element that is already there changes nothing, and `add` returns `false`.

5. Maps

A map stores each *key* with one *value*, and finds a value by its key. Counting is a typical use:

```
Map<String, Integer> plays = new TreeMap<>();
String[] history = {"Static", "Echo Park", "Static", "Slow Burn", "Glass Garden",
        "Static", "Paper Moons", "Echo Park", "Northern Lights"};
for (String title : history) {
    plays.put(title, plays.getOrDefault(title, 0) + 1);
}
```

```
System.out.println(plays);
System.out.println("Static: " + plays.get("Static") + ", Midnight Drive: "
        + plays.get("Midnight Drive") + ", contains Echo Park: "
        + plays.containsKey("Echo Park"));
```

```
{Echo Park=2, Glass Garden=1, Northern Lights=1, Paper Moons=1, Slow Burn=1, Static=3}
Static: 3, Midnight Drive: null, contains Echo Park: true
```

- `put` adds a key, or replaces the value of a key that is already there.
- `get` returns `null` for a key that is not in the map, so `getOrDefault(title, 0)` is safer for counting.
- A map cannot be looped over directly. `entrySet()` gives its entries, `keySet()` its keys, and `values()` its values.

```
for (Map.Entry<String, Integer> entry : plays.entrySet()) {
    System.out.println(entry.getKey() + " played " + entry.getValue() + " times");
}
```

```
Echo Park played 2 times
Glass Garden played 1 times
...
```

The three map classes order their keys the same way the three set classes order their elements. Putting the same history into a `HashMap` and a `LinkedHashMap` gives these keys:

```
HashMap keys:       [Echo Park, Slow Burn, Northern Lights, Paper Moons, Static, Glass Garden]
LinkedHashMap keys: [Static, Echo Park, Slow Burn, Glass Garden, Paper Moons, Northern Lights]
```

The lambda expressions notes show a shorter way to count, with `merge`.

6. equals and hashCode

Collections decide whether two elements are "the same" with `equals`, not `==`. Here is a song written as an ordinary class, with `equals` but no `hashCode`:

```
public class Track {
    private final String title;
    private final String artist;
    ...
    @Override
    public boolean equals(Object other) {
        return other instanceof Track t
                && title.equals(t.title) && artist.equals(t.artist);
    }
}
```

```
Track a = new Track("Static", "Neon Harbor");
Track b = new Track("Static", "Neon Harbor");
System.out.println("equals: " + a.equals(b));
System.out.println("in a list: " + List.of(a).contains(b));

Set<Track> set = new HashSet<>();
set.add(a);
set.add(b);
Track c = new Track("Static", "Neon Harbor");
System.out.println("set size: " + set.size() + ", contains an equal Track: "
        + set.contains(c));
```

```
equals: true
in a list: true
set size: 2, contains an equal Track: false
```

The list found the equal track, because `contains` on a list calls `equals` on each element. The set did not: it kept two equal tracks, and could not find a third. A `HashSet` or `HashMap` first uses `hashCode` to decide where to look, and only then calls `equals`. Without a `hashCode` of its own, each `Track` gets a different hash code from `Object`, so equal tracks are looked for in different places. The compiler warns about this when warnings are on:

```
warning: [overrides] Class Track overrides equals, but neither it nor any superclass overrides
hashCode method
```

The rule: *if you override `equals`, override `hashCode` too*, using the same fields, so that equal objects always have equal hash codes. Eclipse can write both for you (**Source**, then **Generate hashCode() and equals()**). A record writes both automatically, which is one reason records make good elements and keys: with `Song`, the same test leaves the set with one element.

7. Sorted Collections and Comparable

A `TreeSet` or `TreeMap`, and `Collections.sort`, put elements in their *natural order*, the order defined by the elements' `compareTo` method. `String`, the wrapper classes, and `LocalDate` already have one:

```
Collections.sort(titles);
System.out.println(titles);
System.out.println("max: " + Collections.max(titles) + ", min: "
        + Collections.min(titles));
```

```
[Echo Park, Glass Garden, Midnight Drive, Northern Lights, Paper Moons, Slow Burn, Static]
max: Static, min: Echo Park
```

A class of your own gets a natural order by implementing `Comparable`. `Song` does not, and a `TreeSet<Song>` compiles anyway, then fails as soon as it tries to order a song:

```
Exception in thread "main" java.lang.ClassCastException: class Song cannot be cast to class
```

```
java.lang.Comparable (...)
```

Either give the class a natural order, or give the `TreeSet` a `Comparator` that defines the order from outside the class. The lambda expressions notes show how to write one in a single line.

8. Removing While Looping

Removing from a collection inside a for-each loop over that same collection is not allowed:

```
for (Song s : songs) {
    if (s.seconds() > 300) {
        songs.remove(s);                // not safe
    }
}
```

```
Exception in thread "main" java.util.ConcurrentModificationException
```

A for-each loop uses an *iterator* behind the scenes, and the iterator notices that the list changed without it. To remove while looping, use the iterator yourself and call its `remove`:

```
Iterator<Song> it = songs.iterator();
while (it.hasNext()) {
    if (it.next().seconds() < 200) {
        it.remove();                    // safe: the iterator removes it
    }
}
```

That removes the two songs under 200 seconds safely. The lambda expressions notes show an even shorter way, `removeIf`.

9. Collections That Cannot Be Changed

`List.of`, `Set.of`, and `Map.of` create collections that cannot be changed. So does `List.copyOf`, which makes such a copy of another collection. Trying to change one throws an exception:

```
List<String> fixed = List.of("Static", "Echo Park");
fixed.add("Slow Burn");
```

```
Exception in thread "main" java.lang.UnsupportedOperationException
```

A copy and a *view* are different. `Collections.unmodifiableList` returns a view that cannot be changed through the view, but still shows every change to the list underneath:

```
List<String> source = new ArrayList<>(List.of("Static", "Echo Park"));
List<String> copy = List.copyOf(source);
List<String> view = Collections.unmodifiableList(source);
source.add("Slow Burn");
```

```
System.out.println("copy: " + copy + ", view: " + view);
```

```
copy: [Static, Echo Park], view: [Static, Echo Park, Slow Burn]
```

A getter that returns a collection field should usually return `List.copyOf(field)`. Otherwise any caller can change the object's data without going through its methods, which breaks encapsulation.

10. Queues and Deques

A *queue* gives elements back in the order they arrived, first in, first out. A *stack* gives back the newest first, last in, first out. `ArrayDeque` does both:

```
Queue<String> upNext = new ArrayDeque<>();
upNext.offer("Static");
upNext.offer("Echo Park");
upNext.offer("Slow Burn");
System.out.println("plays " + upNext.poll() + ", then " + upNext.peek());

Deque<String> history = new ArrayDeque<>();
history.push("Static");
history.push("Echo Park");
history.push("Slow Burn");
System.out.println("back to " + history.pop() + ", then " + history.pop());
```

```
plays Static, then Echo Park
back to Slow Burn, then Echo Park
```

`offer` adds to the back of a queue, `poll` removes from the front, and `peek` looks without removing. `push` and `pop` add and remove at the top of a stack. Java also has an older `Stack` class; `ArrayDeque` replaces it.

11. Choosing a Collection

Table 3: Which collection to use.

If the code needs to	Use
Keep elements in order, by position	`ArrayList`
Keep one of each, in any order	`HashSet`
Keep one of each, in the order added	`LinkedHashSet`
Keep one of each, sorted	`TreeSet`
Look up a value by a key	`HashMap`
Look up by key, with keys in the order added	`LinkedHashMap`
Look up by key, with keys sorted	`TreeMap`
Take elements first in, first out, or last in, first out	`ArrayDeque`
Hand out a collection that must not change	`List.of, List.copyOf`

Two rules apply to all of them: a *hash* collection needs `equals` and `hashCode`, and a *tree* collection needs a natural order or a `Comparator`.

12. Try It

1. Make a `Map<String, Integer>` of each artist's total seconds, using `getOrDefault`. Try it with a `HashMap` and then a `TreeMap`.
2. Add a `hashCode` method to `Track` that uses `Objects.hash(title, artist)`, and run the set test again. What is the set's size now?
3. Write a getter that returns a `List` field directly, change the list through the getter, and print the object. Then return `List.copyOf` instead, and try again.
4. Predict the output of `lengths.remove(0)` on a `List<Integer>` holding `[0, 1, 2]`. Then run it.